

Machine Program: Procedure

Jinyang Li

What we've learnt about how hardware runs a program?

- Basic hardware execution:
 - CPU fetch next instructions from memory according to %rip
 - Decode and execute instruction (e.g. mov, add)
 - CPU updates %rip to point to next instruction
- Modes of execution:
 - Sequential:
 - PC (%rip) is changed to point to the next instruction
 - Control flow: jmp, conditional jmp
 - PC (%rip) is changed to point to the jump target address

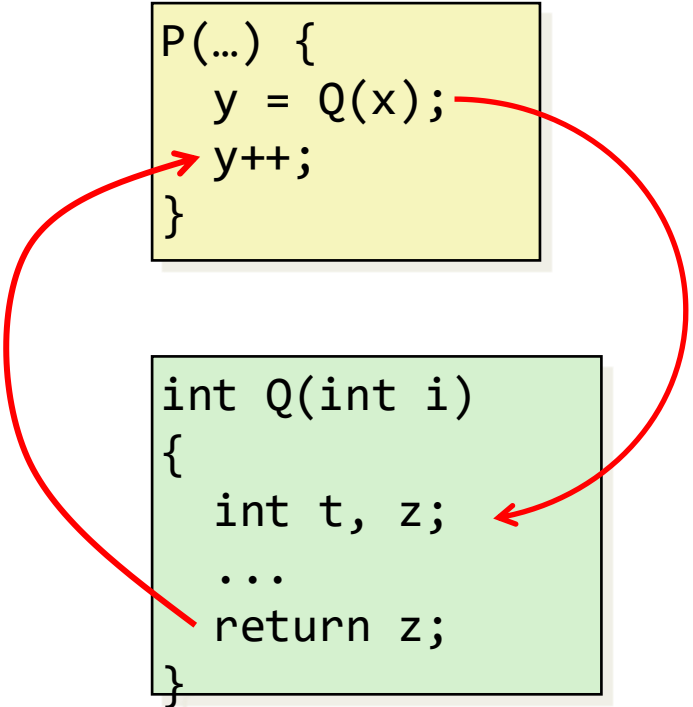
Today's lesson plan

- How x86 supports function call
 - Role of stack
 - Call / ret
 - Calling convention (where args/ret-vals are stored)

Requirements of procedure calls?

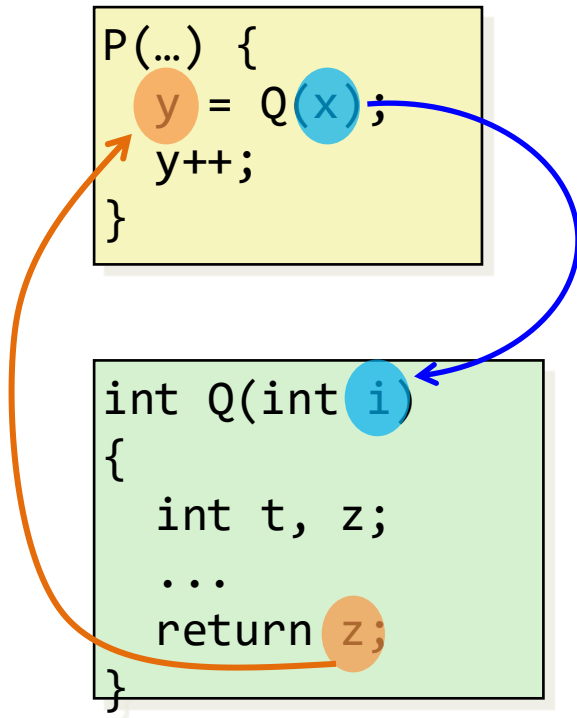
1. Passing control

```
P(...) {  
  y = Q(x);  
  y++;  
}
```



```
int Q(int i)  
{  
  int t, z;  
  ...  
  return z;  
}
```

Requirements of procedure calls?



1. Passing control
2. Passing Arguments & return value

Requirements of procedure calls?

```
P(...) {  
  y = Q(x);  
  y++;  
}
```

```
int Q(int i)  
{  
  int t, z;  
  ...  
  return z;  
}
```

1. Passing control
2. Passing Arguments & return value
3. Allocate / deallocate local variables

How to transfer control for procedure calls?

```
void main(){  
    ..  
    f(..)  
L1: ..  
}
```

```
void f(){  
    ..  
    g(..)  
L2: ..  
}
```

```
void g(){  
    ..  
    h(..)  
L3: ..  
}
```

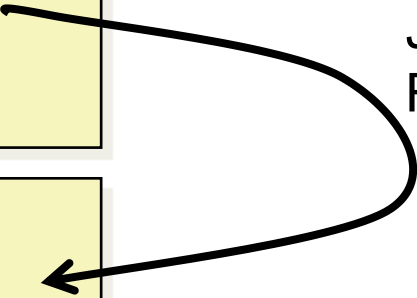
How to transfer control for procedure calls?

```
void main(){  
    ..  
    f(..)  
L1: ..  
}
```

```
void f(){  
    ..  
    g(..)  
L2: ..  
}
```

```
void g(){  
    ..  
    h(..)  
L3: ..  
}
```

Jump to f()
Remember where to come back



L1

How to transfer control for procedure calls?

```
void main(){  
    ..  
    f(..)  
L1: ..  
}
```

Jump to f()
Remember where to come back

```
void f(){  
    ..  
    g(..)  
L2: ..  
}
```

Jump to g()
Remember where to come back

```
void g(){  
    ..  
    h(..)  
L3: ..  
}
```

L2

L1

How to transfer control for procedure calls?

```
void main(){  
    ..  
    f(..)  
L1: ..  
}
```

Jump to f()
Remember where to come back

```
void f(){  
    ..  
    g(..)  
L2: ..  
}
```

Jump to g()
Remember where to come back

```
void g(){  
    ..  
    h(..)  
L3: ..  
}
```

Jump to h()
Remember where to come back



How to transfer control for procedure calls?

```
void main(){  
    ..  
    f(..)  
L1: ..  
}
```

Jump to f()
Remember where to come back

```
void f(){  
    ..  
    g(..)  
L2: ..  
}
```

Jump to g()
Remember where to come back

```
void g(){  
    ..  
    h(..)  
L3: ..  
}
```

Jump to L3
Forget L3



How to transfer control for procedure calls?

```
void main(){  
    ..  
    f(..)  
L1: ..  
}
```

```
void f(){  
    ..  
    g(..)  
L2: ..  
}
```

```
void g(){  
    ..  
    h(..)  
L3: ..  
}
```

Jump to f()
Remember where to come back

Jump to L2
Forget L2

Jump to L3
Forget L3



How to transfer control for procedure calls?

```
void main(){  
    ..  
    f(..)  
L1: ..  
}
```

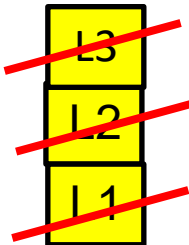
Jump to L1
Forget L1

```
void f(){  
    ..  
    g(..)  
L2: ..  
}
```

Jump to L2
Forget L2

```
void g(){  
    ..  
    h(..)  
L3: ..  
}
```

Jump to L3
Forget L3



How to transfer control for procedure calls?

```
void main(){  
    ..  
    f(..)  
L1: ..  
}
```

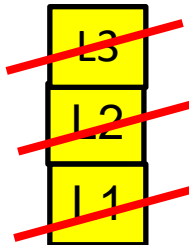
Jump to L1
Forget L1

```
void f(){  
    ..  
    g(..)  
L2: ..  
}
```

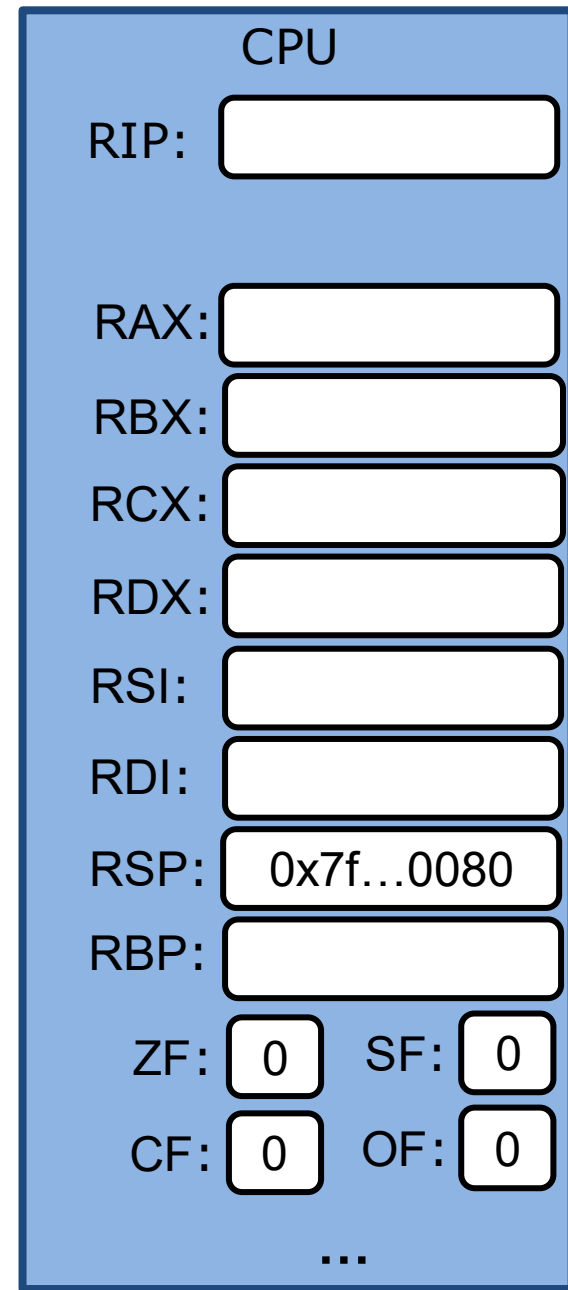
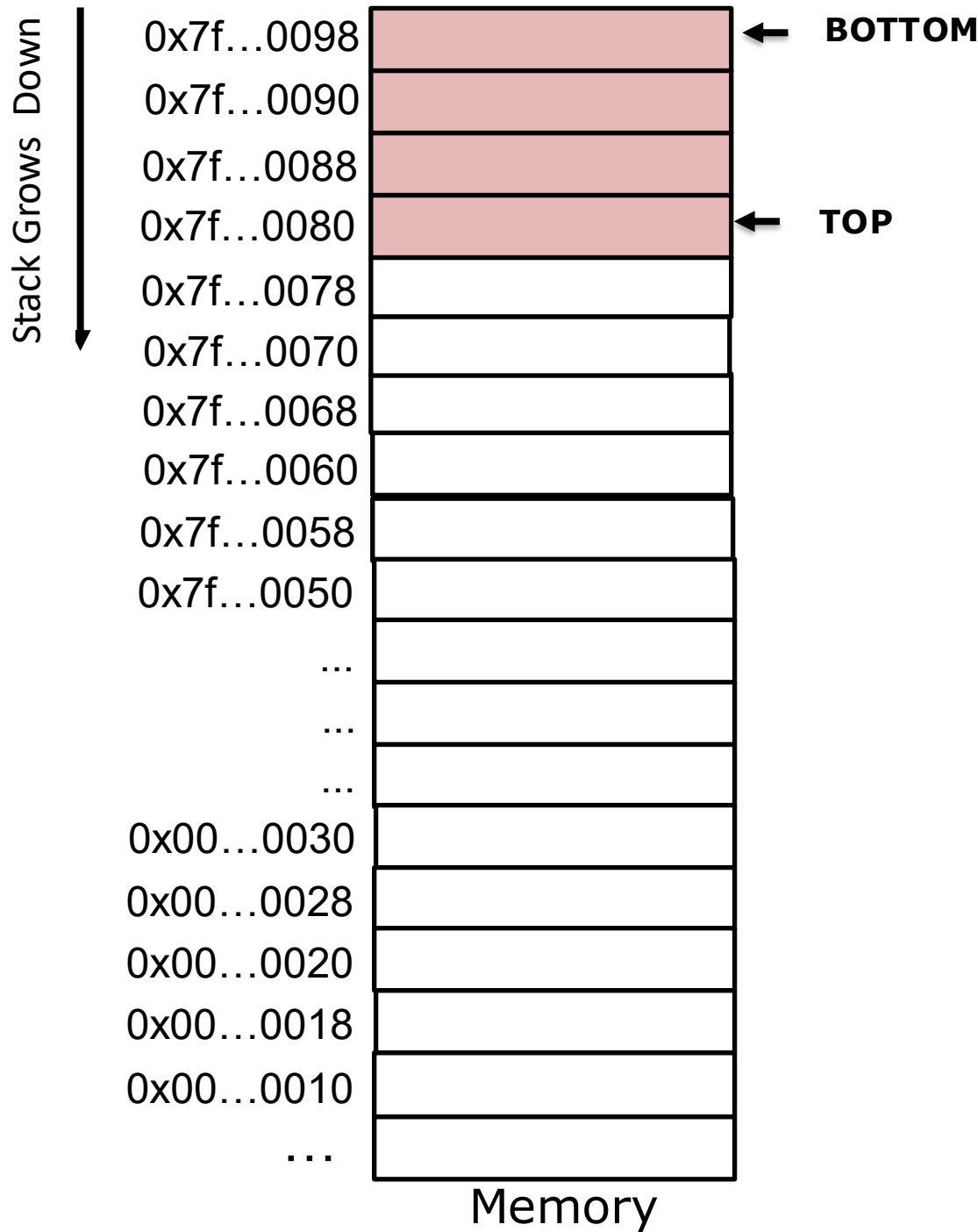
Jump to L2
Forget L2

```
void g(){  
    ..  
    h(..)  
L3: ..  
}
```

Jump to L3
Forget L3



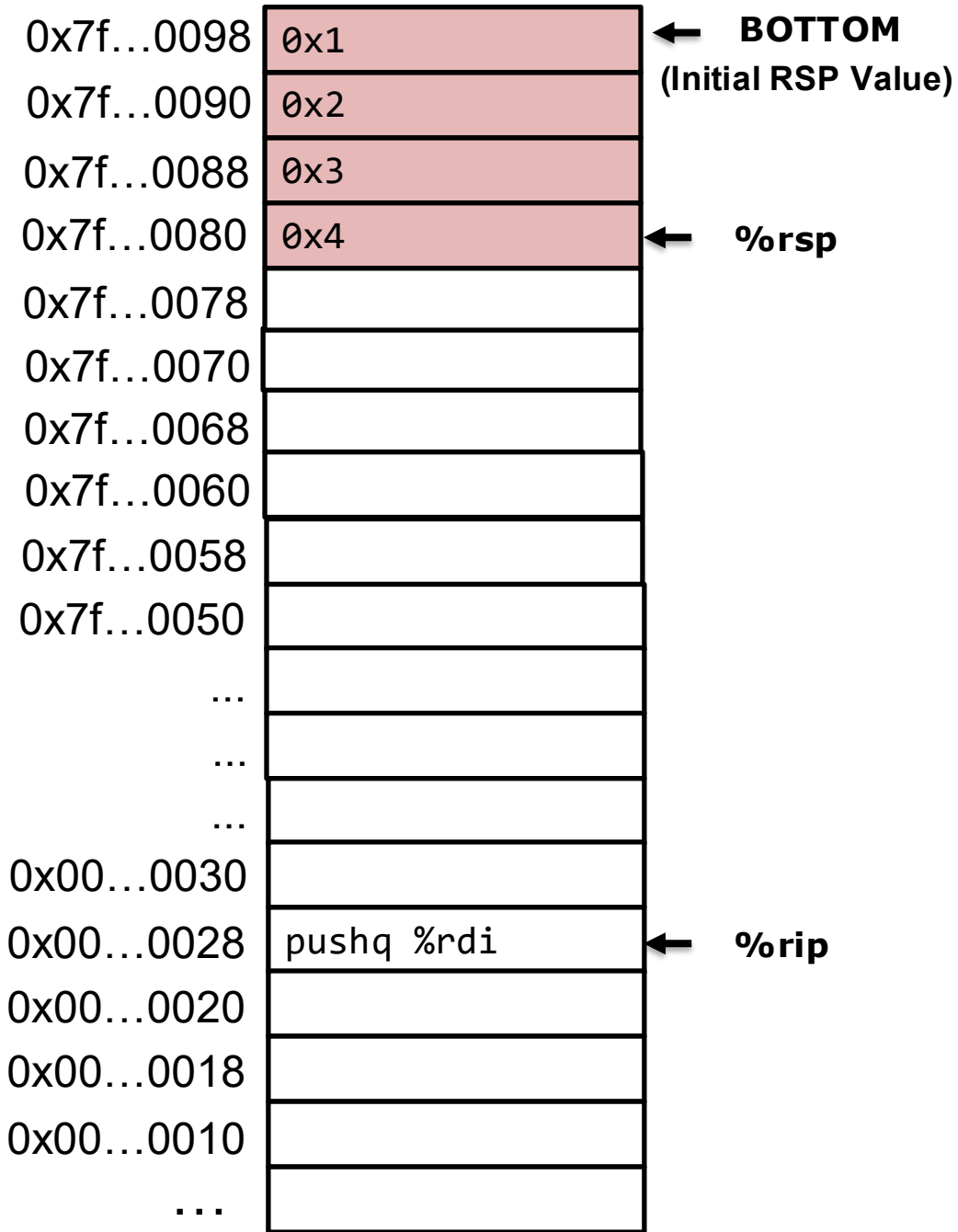
Stack



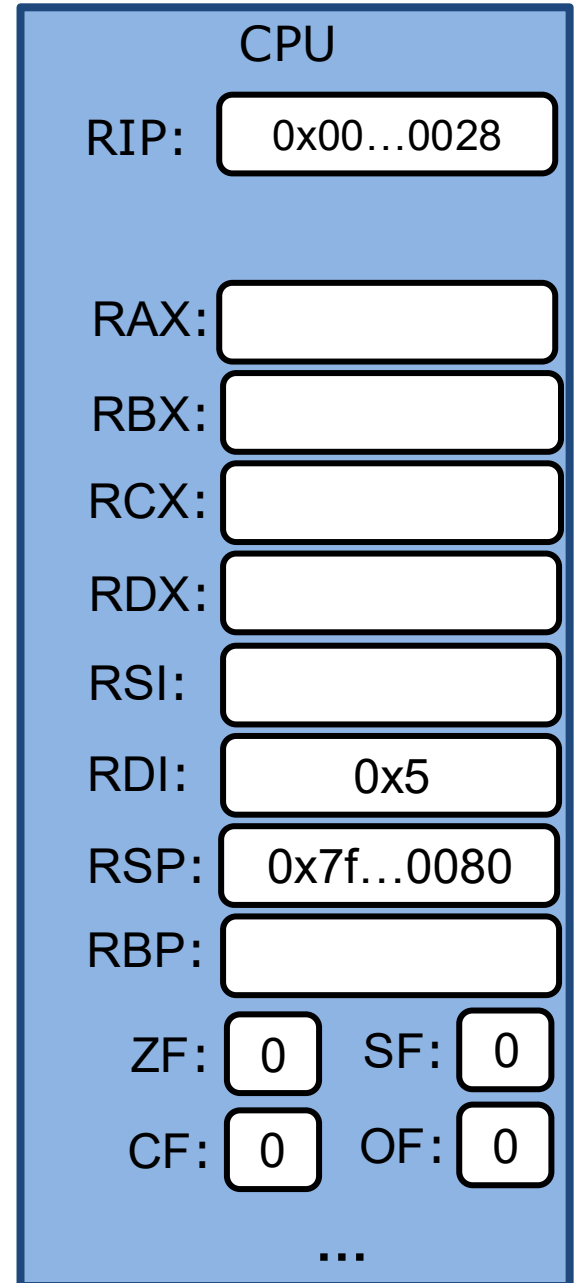
Stack – push Instruction

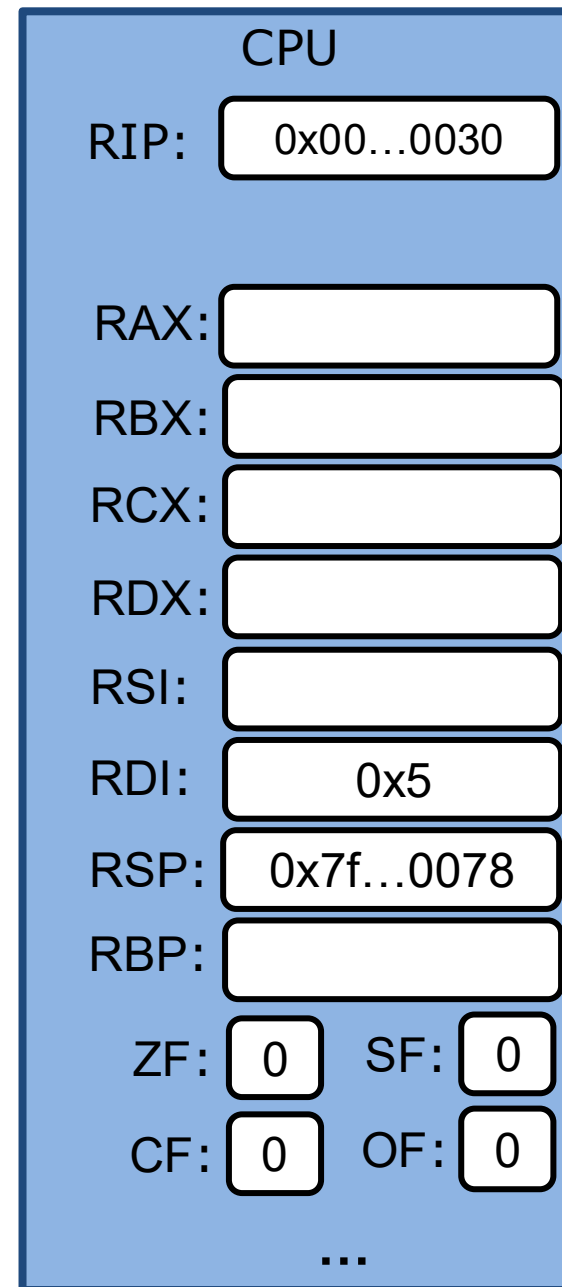
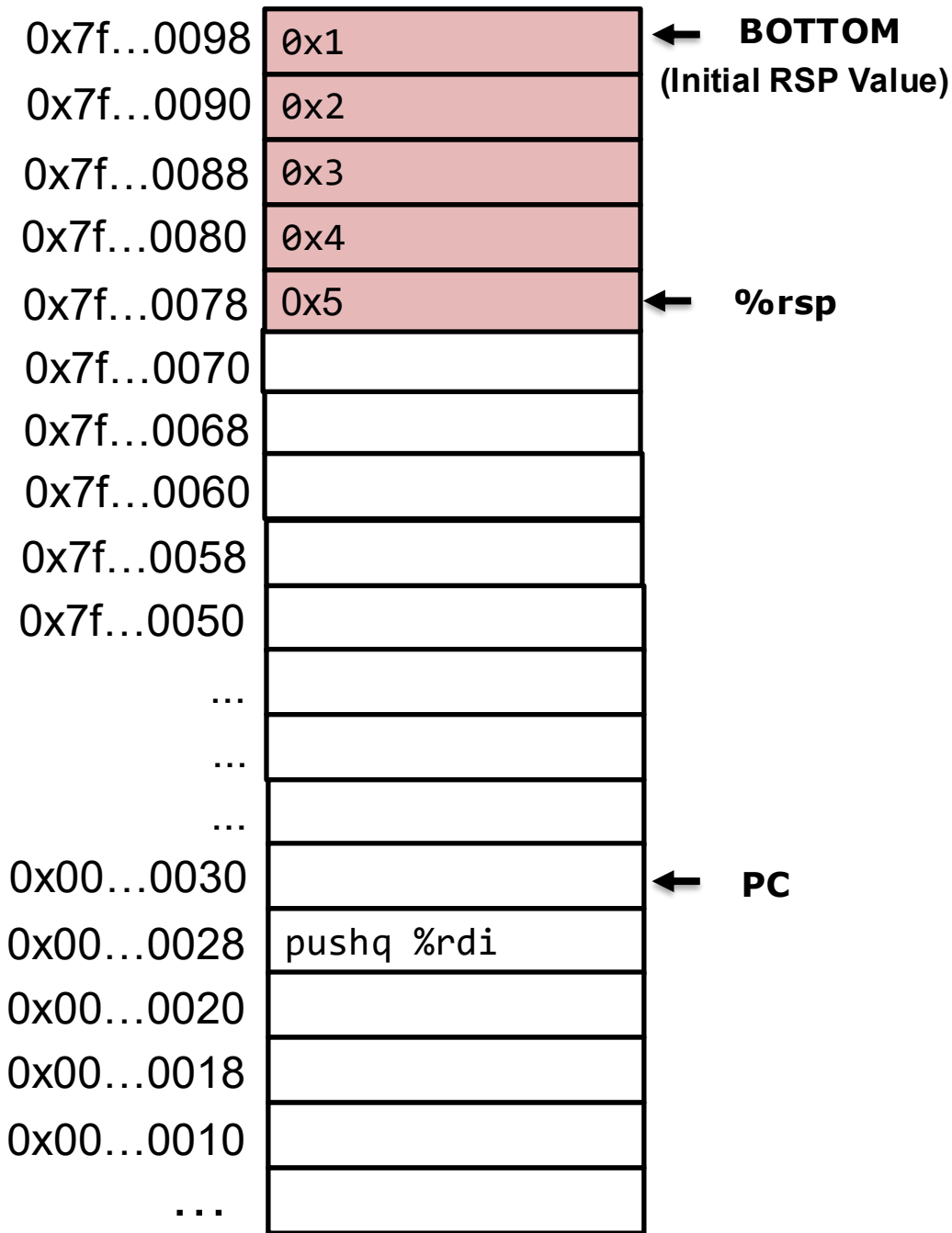
pushq src

- Decrement %rsp by 8
- Write operand at address given by %rsp



Memory

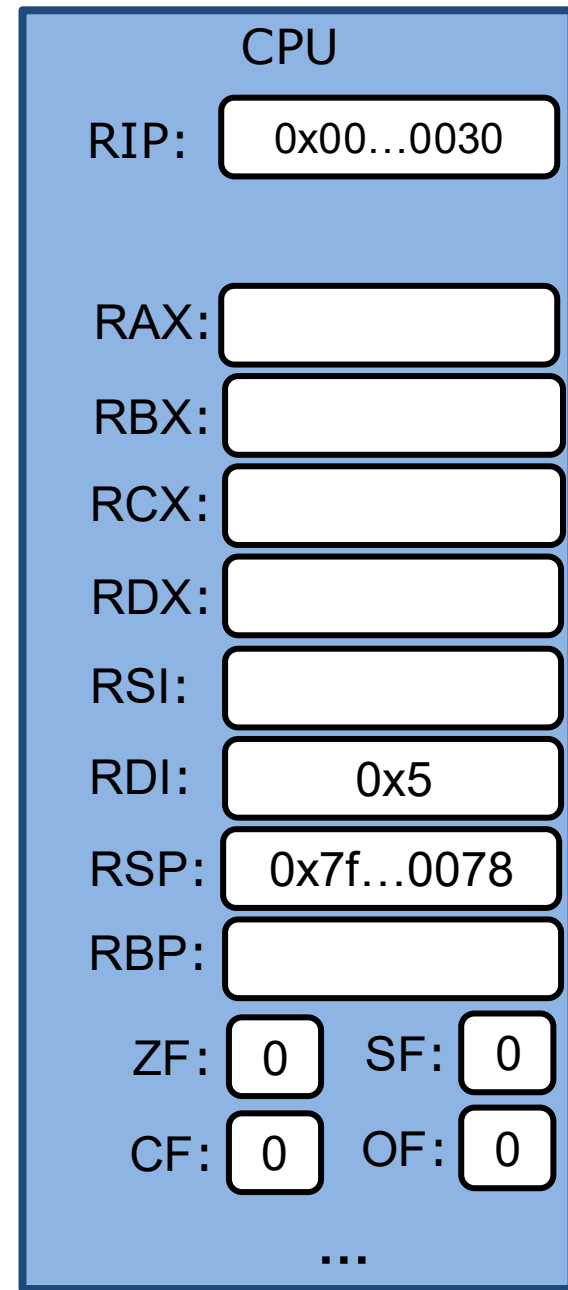
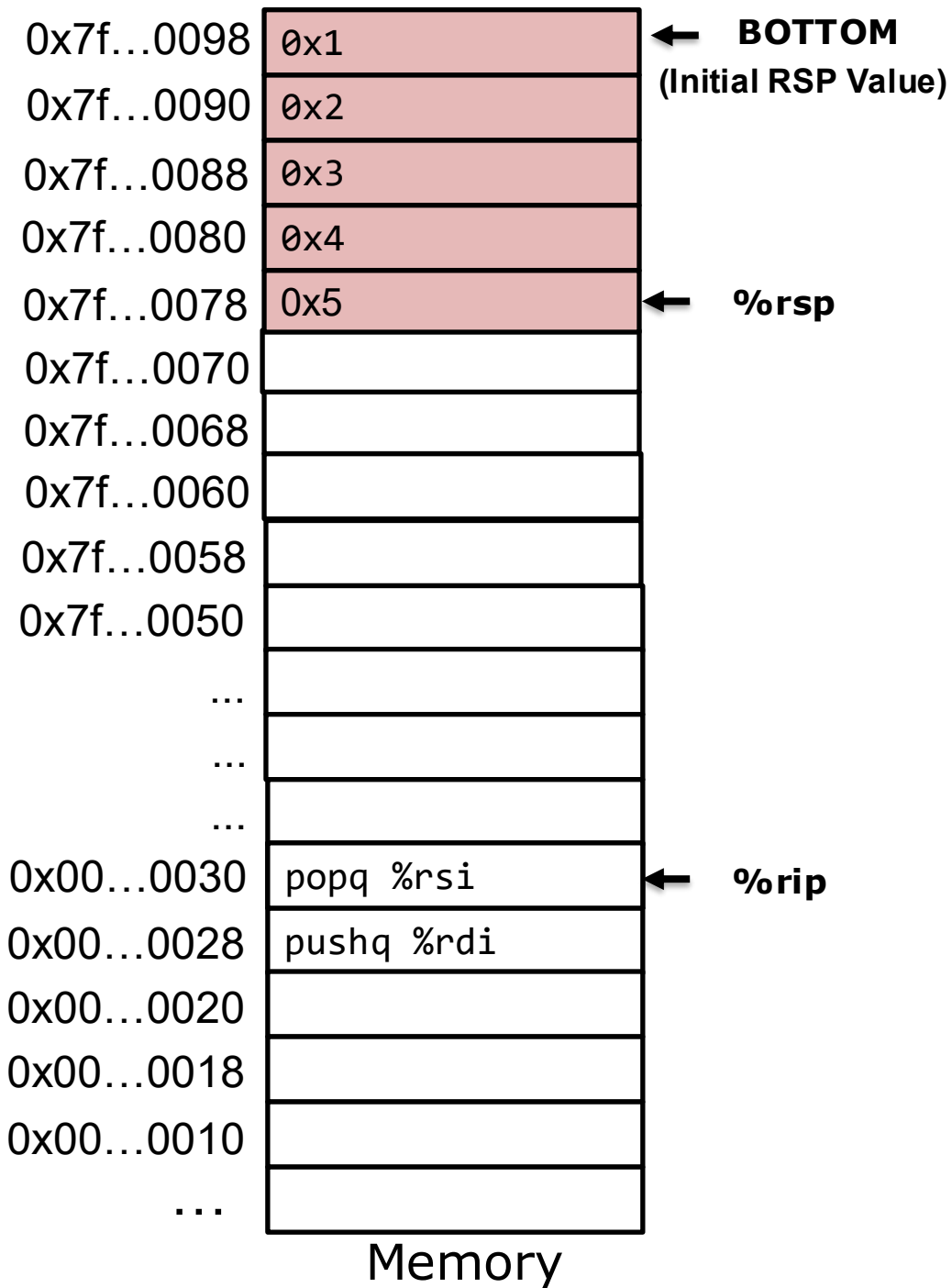


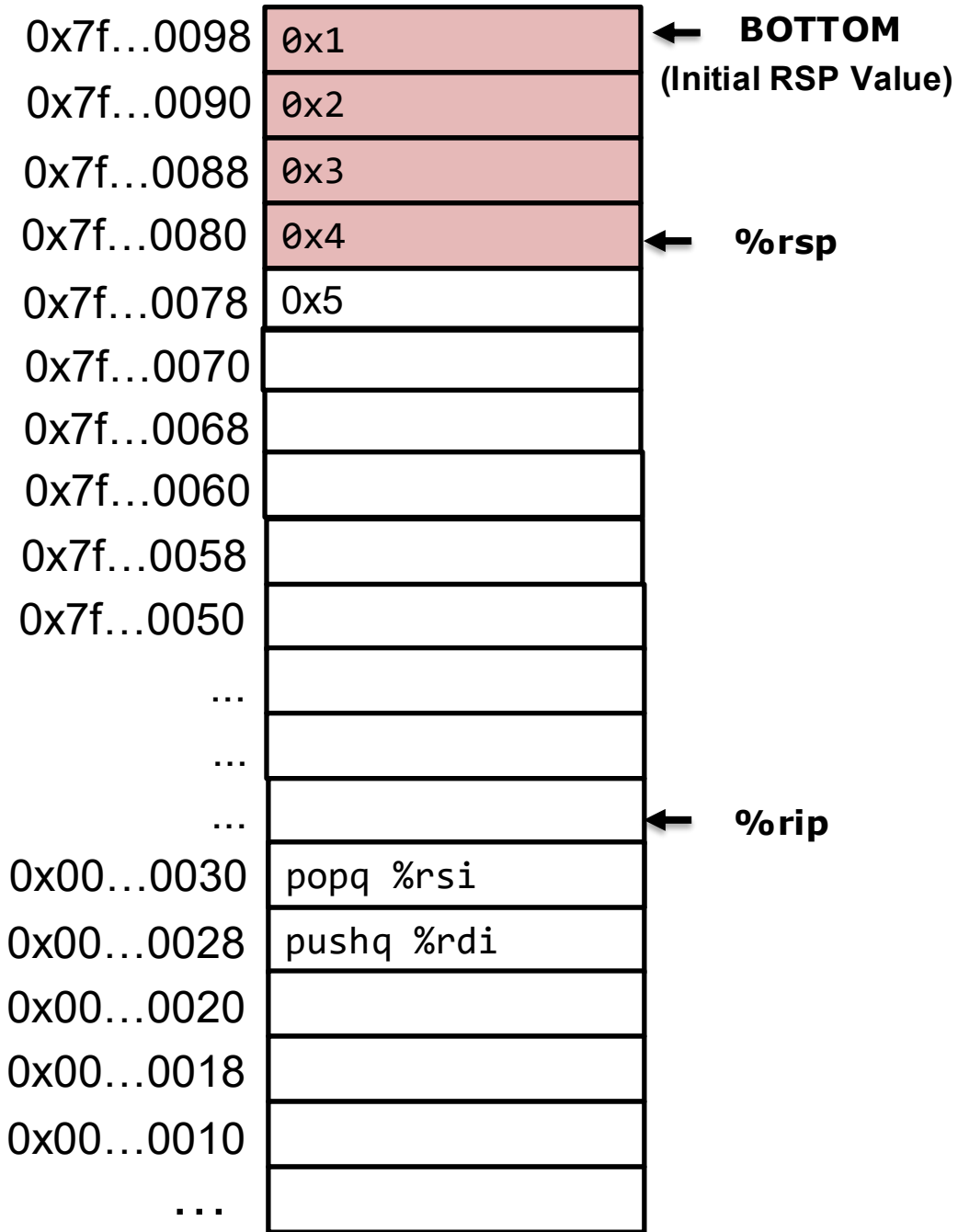


Stack – pop Instruction

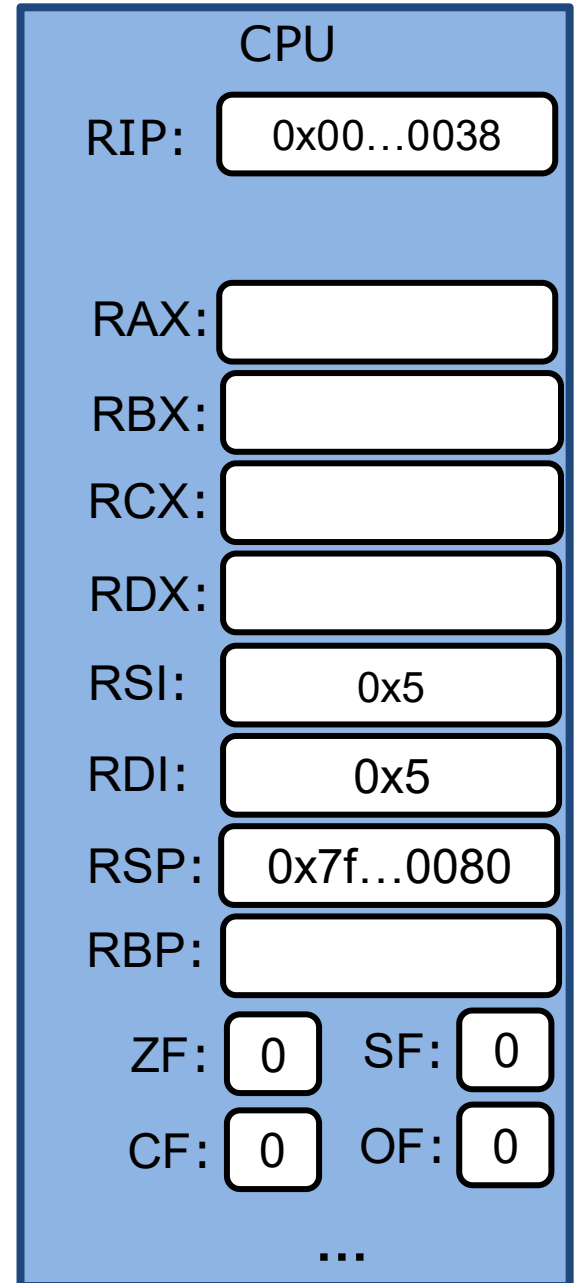
popq dest

- Store the value at address `%rsp` to `dest`
- Increment `%rsp` by 8





Memory



call/ret : control transfer from caller to callee and vice versa

call label

- Push return address on stack
 - return_address = the instruction immediately after **call**
 - `%rsp=%rsp-8, mem[%rsp]=return_addr`
- Jump to the address of the label
 - Label points to the first instruction of the function

ret

- Pop 8 bytes from the stack to %rip
 - `%rip = mem[%rsp], %rsp = %rsp +8`

call/ret : control transfer from caller to callee and vice versa

```
int count = 0;

void inc() {
    count++;
}

int main() {
    inc();
}
```

gcc -Og -S test.c



```
inc:
    addl $0x1, count(%rip)
    ret

main:
    ...
    call    add
    movl $0, %eax
    ...
```

return address points to this instruction

Call instruction: control transfer from caller to callee

```
gcc -Og test.c  
objdump -d a.out
```

```
int count = 0;  
  
void inc() {  
    count++;  
}
```

```
int main() {  
    inc();  
}
```

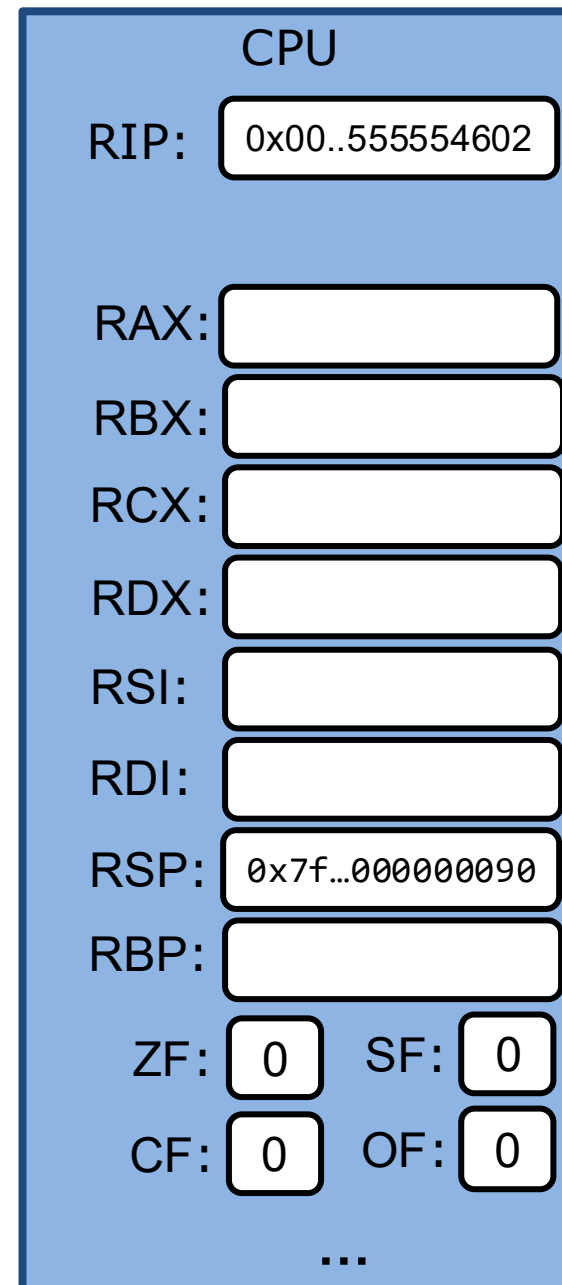
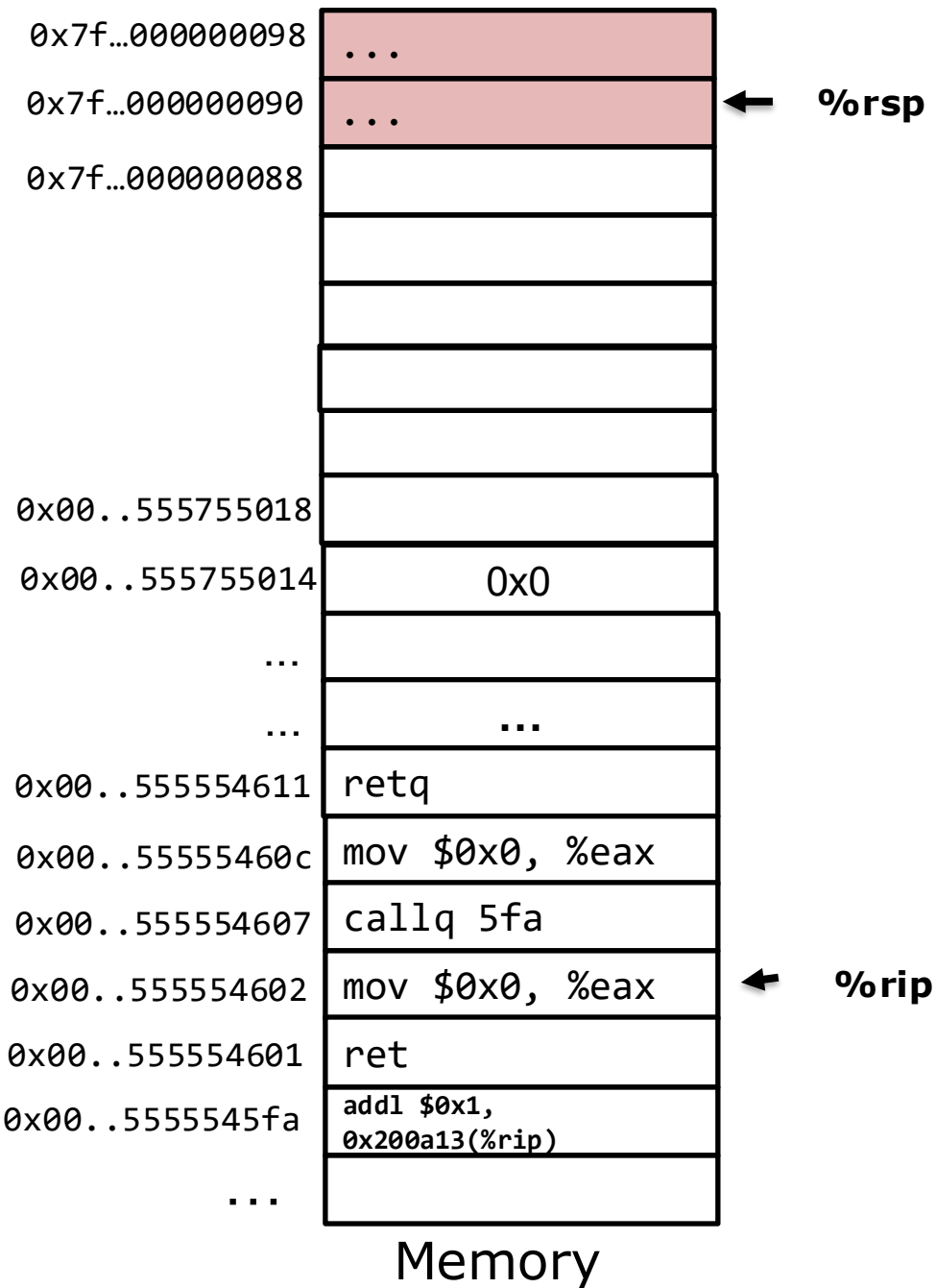


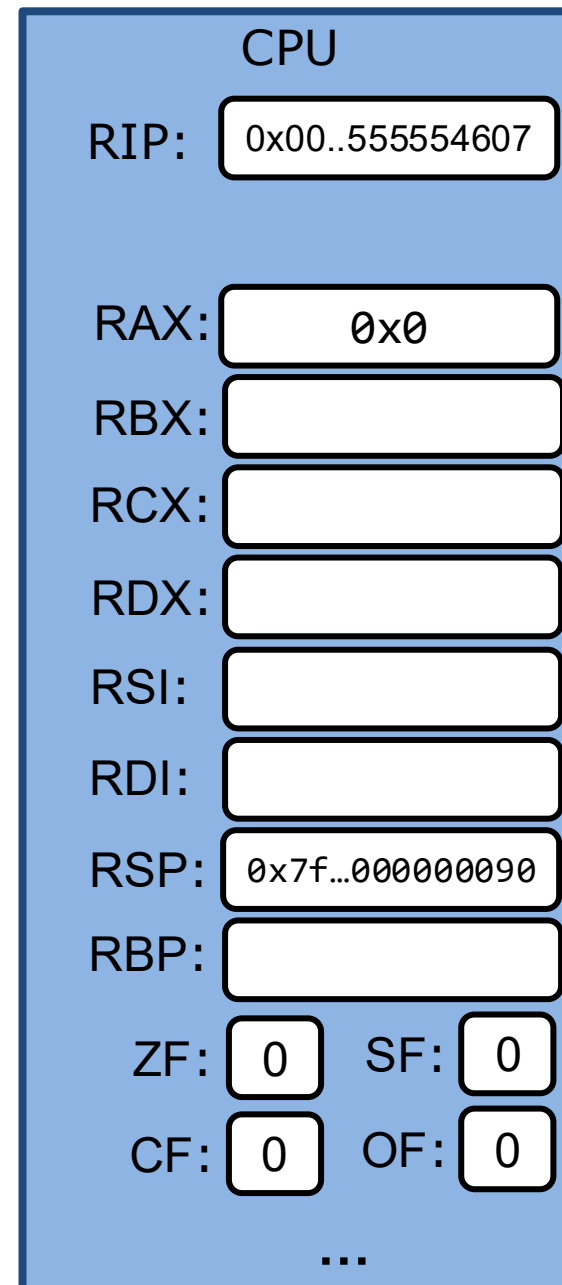
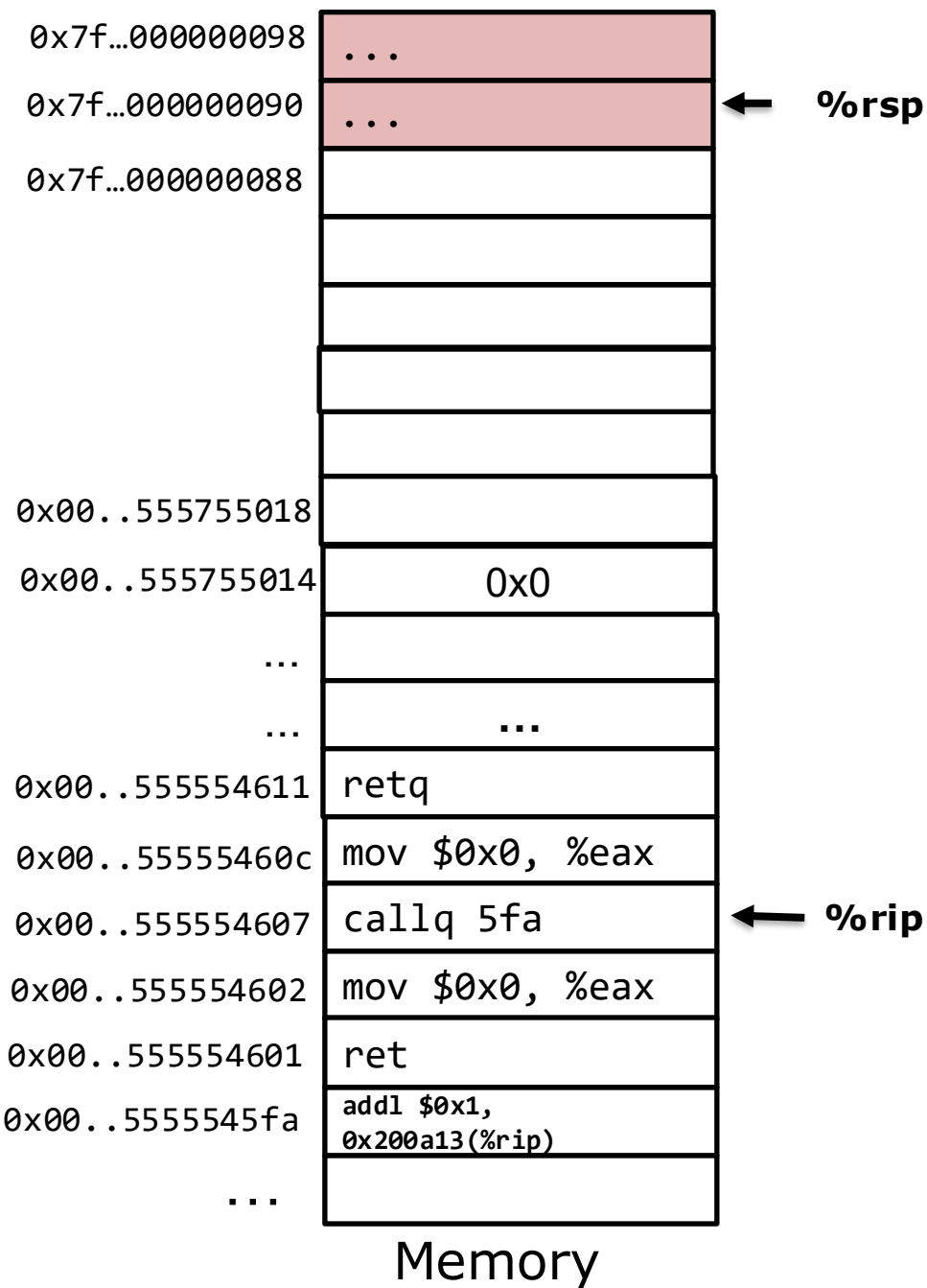
00000000000005fa <inc>:

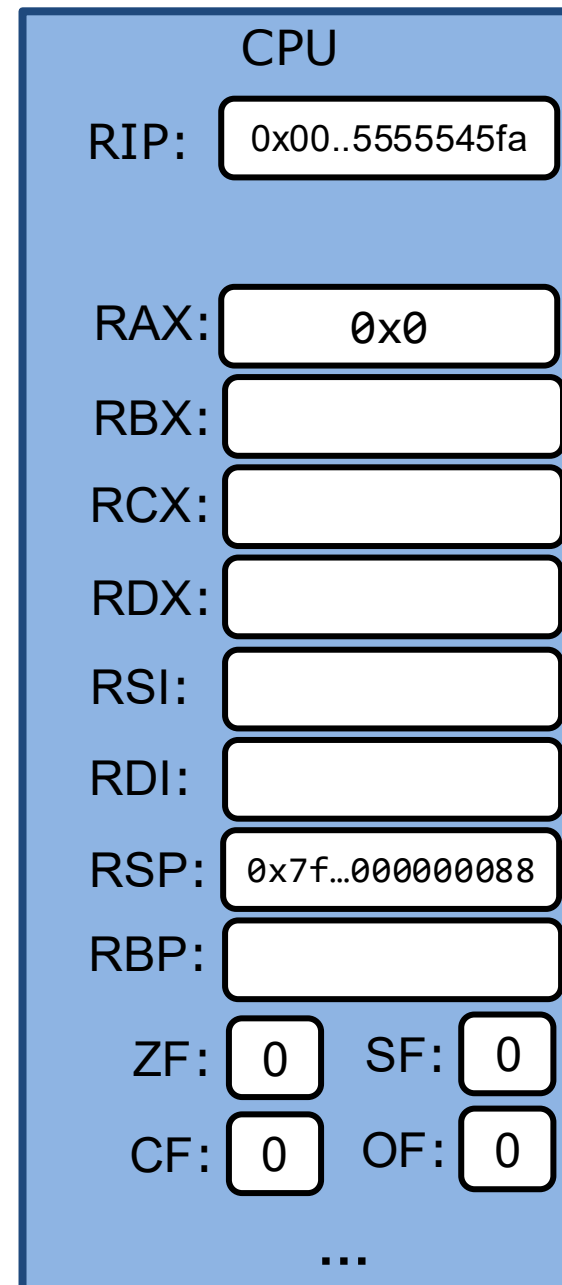
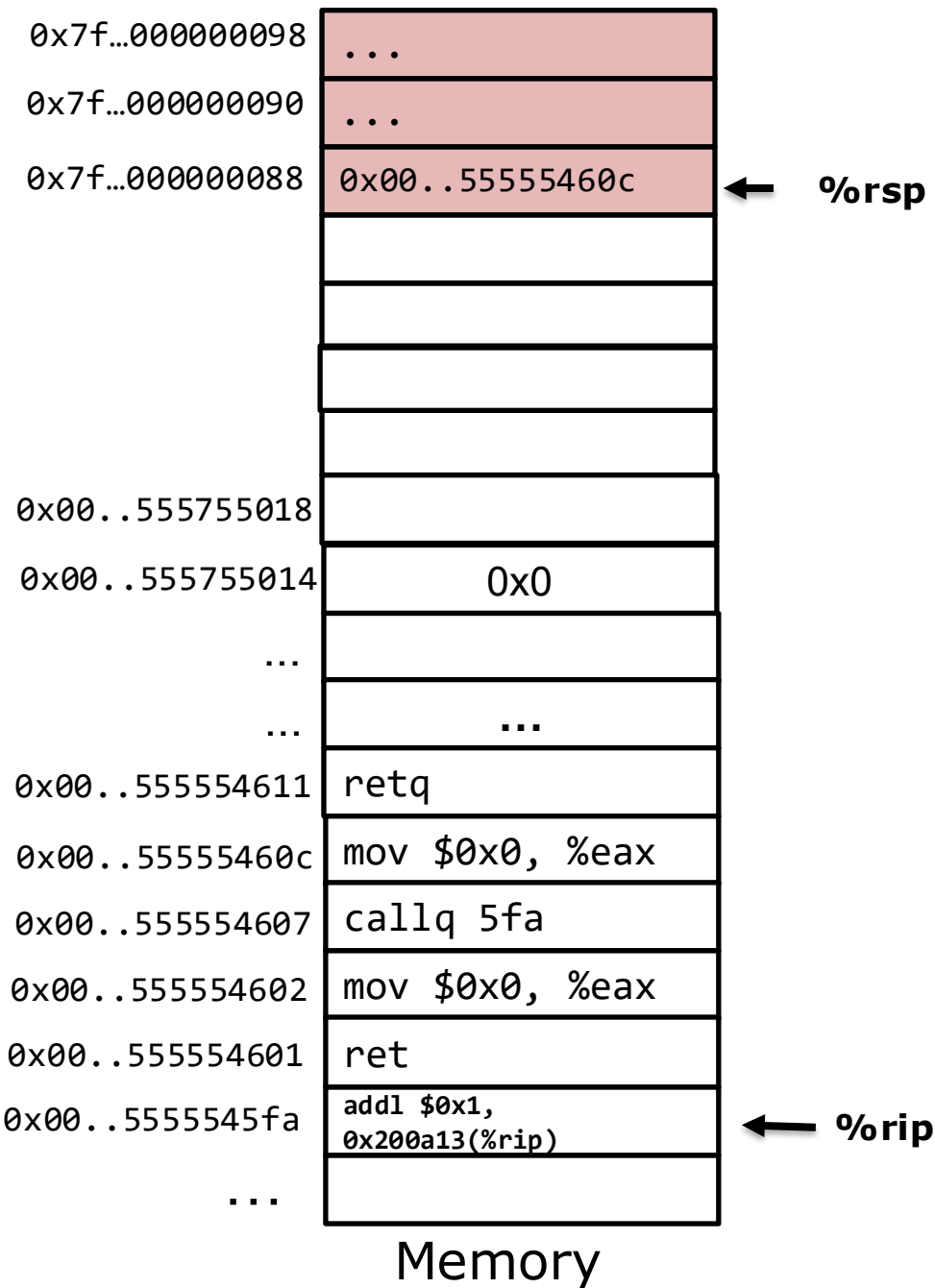
5fa: 83 05 13 0a 20 00 01 addl \$0x1, 0x200a13(%rip)
601: c3 retq

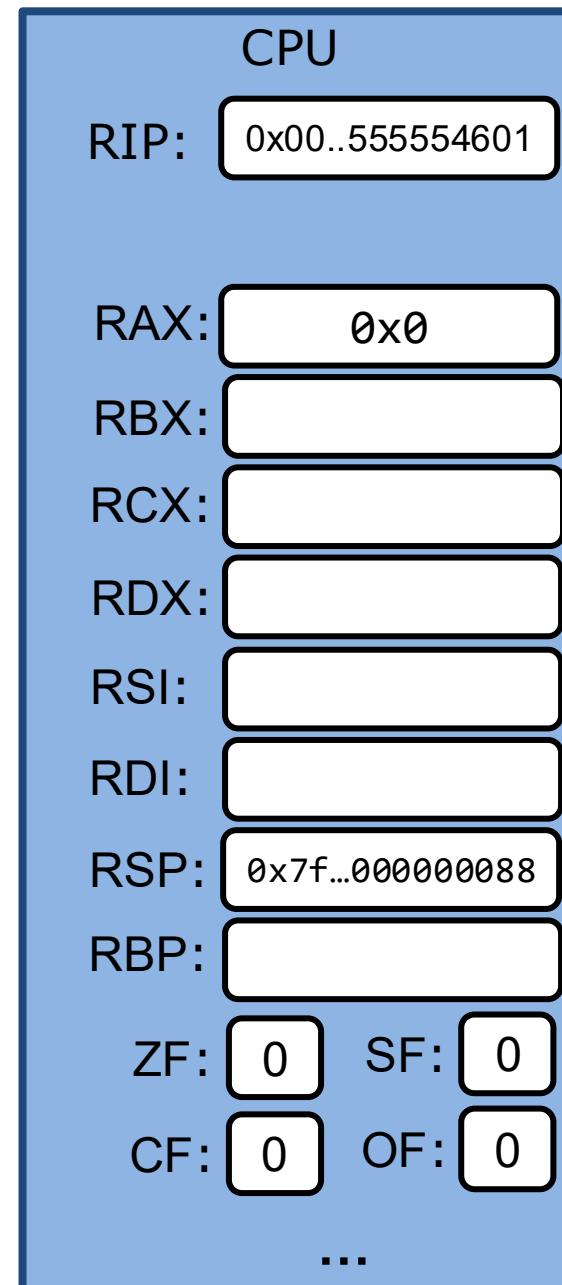
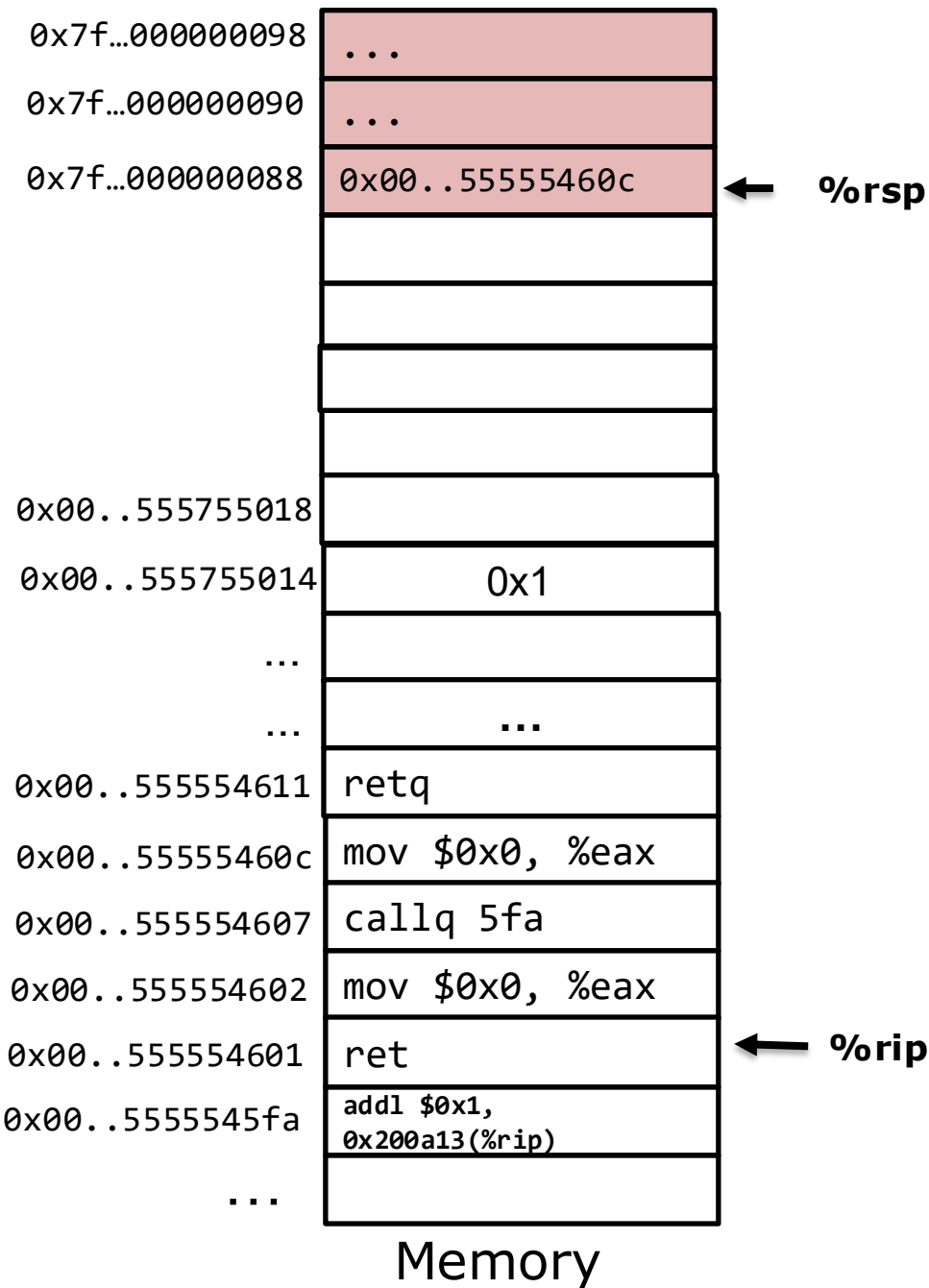
0000000000000602 <main>:

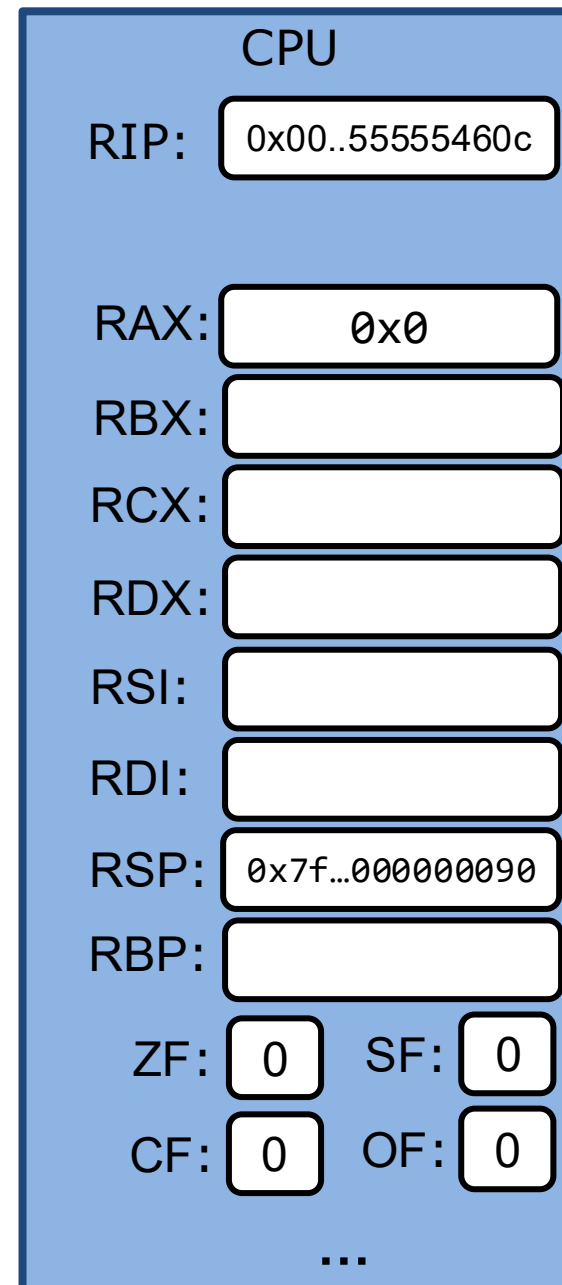
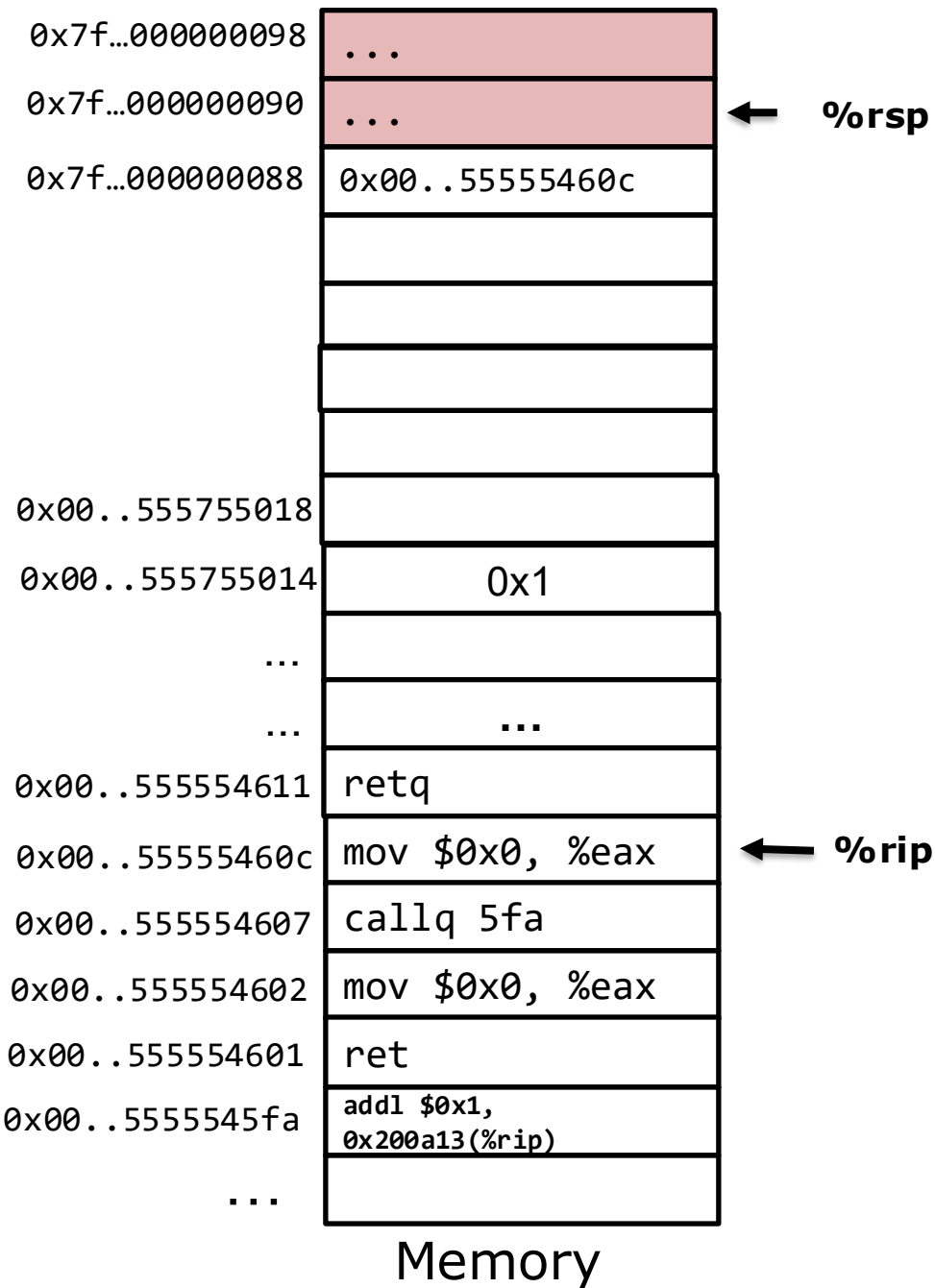
602: b8 00 00 00 00 mov \$0x0,%eax
607: e8 ee ff ff ff callq 5fa <inc>
60c: b8 00 00 00 00 mov \$0x0,%eax
611: c3 retq

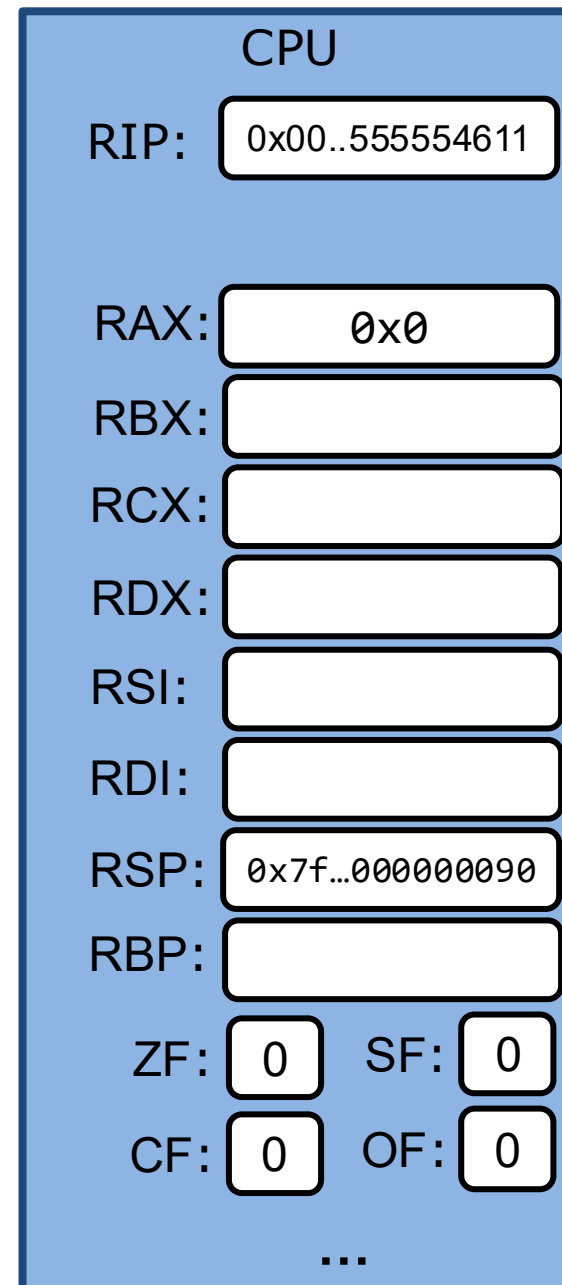
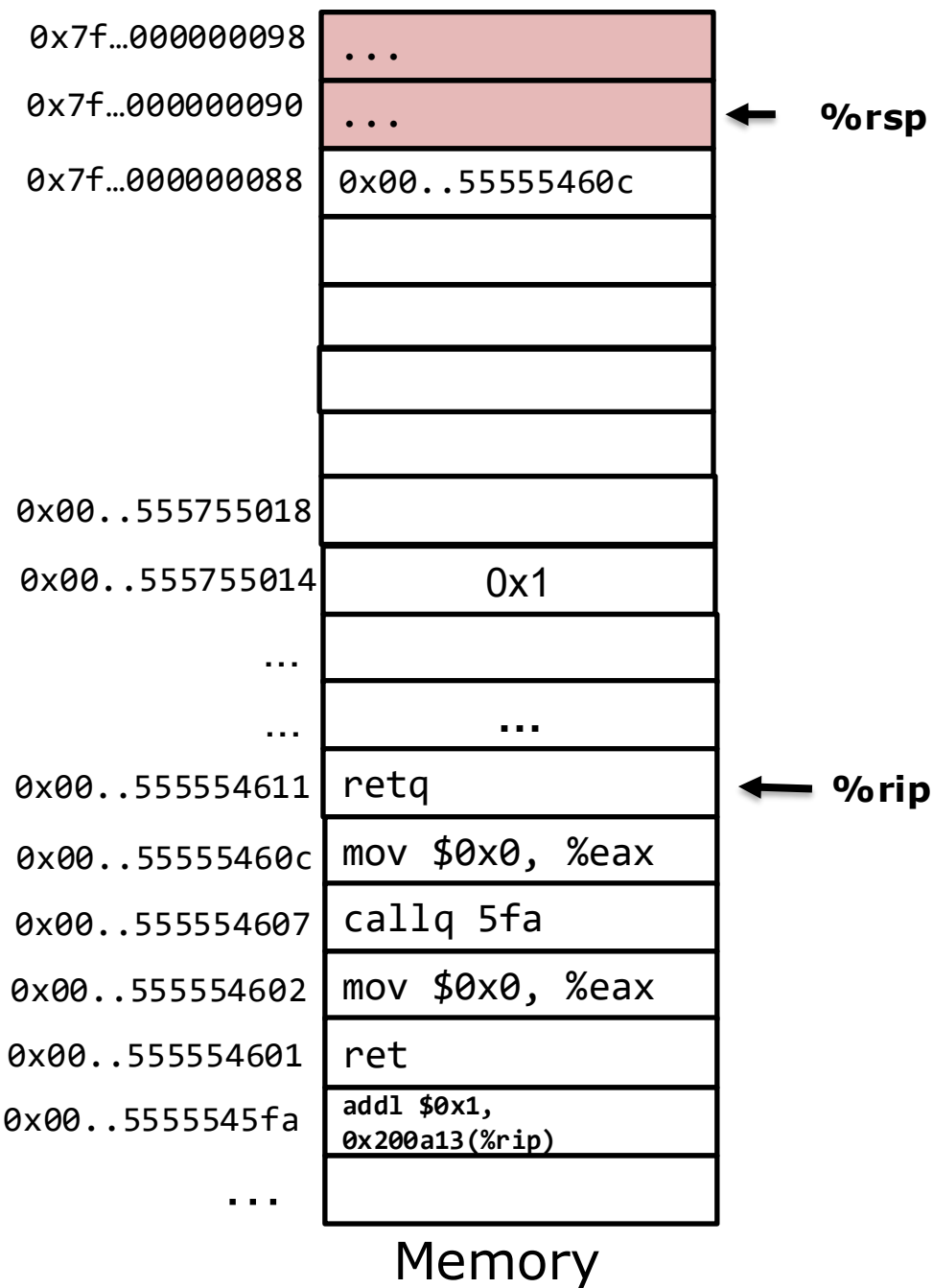












Where to store function arguments and return values?

- Hardware doesn't care where args/return vals are stored
 - It's a software convention
- Design consideration: where to put args and return vals?
 - Arguments and return value are allocated when function is called, de-allocated when function returns.
 - Must do such allocation/de-allocation very fast

Where to store function arguments and return values?

- Two possible designs:
 - Store on stack
 - Store in registers
- The chosen design → the calling convention
 - All code on a computer system must obey the same convention
 - Otherwise, libraries won't work

Registers are much faster than memory
but there are only a few of them



C/UNIX/MacOS's calling convention

Windows have a different calling convention

Registers

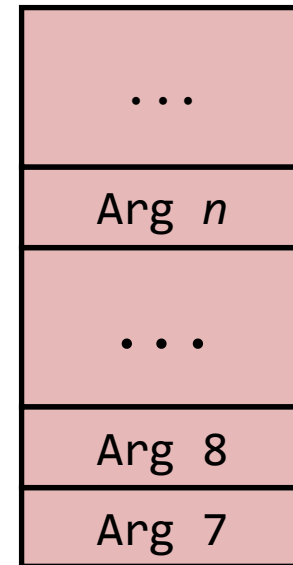
First 6 arguments

%rdi
%rsi
%rdx
%rcx
%r8
%r9

Return value

%rax

Stack



← Stack top

Only allocate stack space when needed

Calling convention: args, return vals

```
int add(int a, int b, int c, int d, int e, int f, int g, int h) {  
    int r = a + b + c + d + e + f + g + h;  
    return r;  
}  
  
int main() {  
    int c = add(1, 2, 3, 4, 5, 6, 7, 8);  
    printf("%d\n", c);  
    return 0;  
}
```

main:

```
pushl    $8  
pushl    $7  
movl     $6, %r9d  
movl     $5, %r8d  
movl     $4, %ecx  
movl     $3, %edx  
movl     $2, %esi  
movl     $1, %edi  
call     add
```

add:

```
addl     %esi, %edi  
addl     %edi, %edx  
addl     %edx, %ecx  
addl     %r8d, %ecx  
addl     %r9d, %ecx  
movl     %ecx, %eax  
addl     8(%rsp), %eax  
addl     12(%rsp), %eax  
ret
```

8(%rsp) stores g

12(%rsp) stores h
what does (%rsp) store?

How do C compilers allocate/deallocate local vars?

- For primitive data types, use registers whenever possible
- Allocate local array/struct variables on the stack

```
int main() {  
    int a[10];  
    clear_array(a, 10);  
    return 0;  
}
```



main:

```
    subq    $48, %rsp  
    movl    $10, %esi  
    movq    %rsp, %rdi  
    call    clear_array  
    movl    $0, %eax  
    addq    $48, %rsp  
    ret
```

array
allocation

array
de-allocation

Calling convention:

Caller vs. callee-save registers

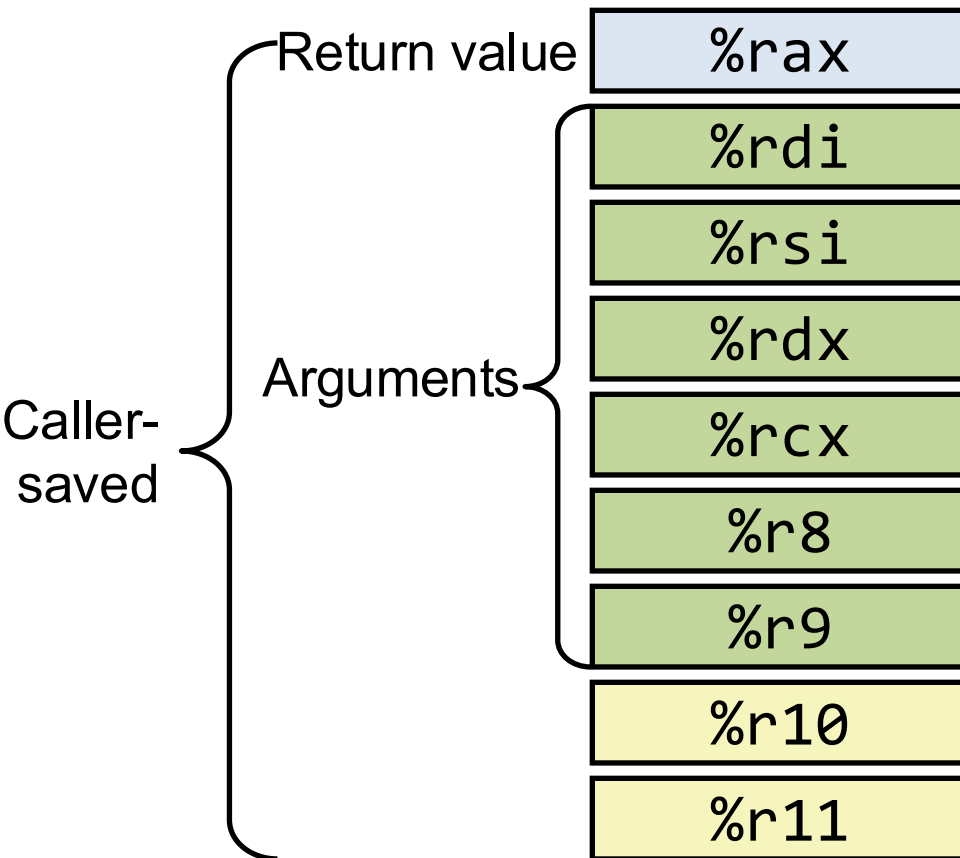
- What can the caller assume about the content of a register across function calls?

```
int foo() {  
    int a;    // suppose a is stored in %r12  
    a = .... // compute result of a  
  
    int r = bar();  
  
    int result = r + a; // does %r12 still store the value of a?  
    return result;  
}
```

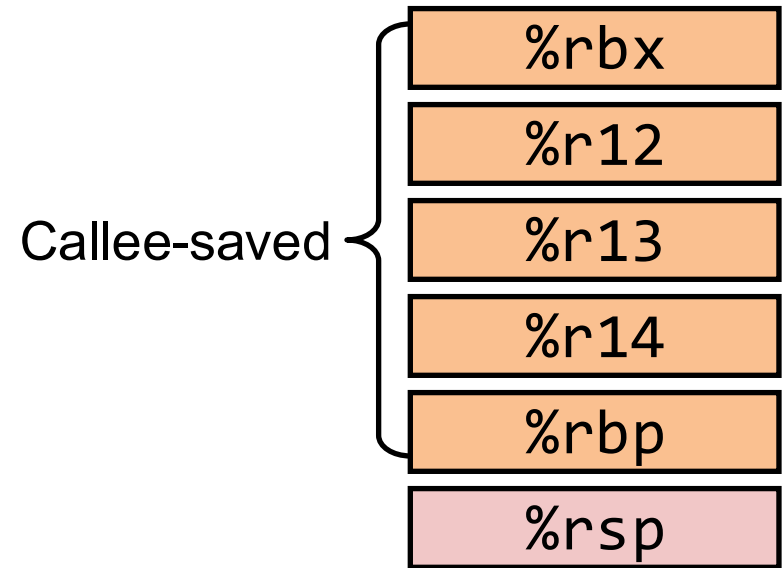
Calling convention: register saving

- Caller saved
 - If caller is going to need X's value after the call, it saves X on stack before the call and restores X after the call
- Callee saved
 - If callee is going to use Y, it saves Y on stack before using and restores Y before returning to caller

Calling convention: Register saving



Callee can directly use these registers



Caller can assume callee-save registers are unchanged across function calls

Example

```
int add2(int a, int b)
{
    return a + b;
}
```

```
int add3(int a, int b, int c)
{
    int r = add2(a, b);
    r = r + c;
    return r;
}
```

```
add2:
    leal    (%rdi,%rsi), %eax
    ret
```

```
add3:
    pushq   %rbx
    movl    %edx, %ebx
    call    add2
    addl    %ebx, %eax
    popq    %rbx
    ret
```

Registers

First 6 Arguments: %rdi, %rsi, %rdx, %rcx, %r8, %r9

Return value: %rax

Example

```
int add2(int a, int b)
{
    return a + b;
}
```

```
int add3(int a, int b, int c)
{
    int r = add2(a, b);
    r = r + c;
    return r;
}
```

*%rdx (contains c) is caller save,
i.e. may be changed by add2*

```
add2:
    leal    (%rdi,%rsi), %eax
    ret
```

*save %rbx (callee-save)
before overwriting it*

```
add3:
    pushq   %rbx
    movl    %edx, %ebx
    call    add2
    addl    %ebx, %eax
    popq    %rbx
    ret
```

*c is copied to %ebx,
which is callee save*

restore %rbx before ret

Registers

First 6 Arguments: %rdi, %rsi, %rdx, %rcx, %r8, %9

Return value: %rax

Summary

- Function call in x86
 - Stack (stores return-address, local variables)
 - Push, pop
 - Call/ret
 - Call saves return-address on stack, ret pops return-address from stack
 - UNIX calling convention
 - First 6 function arguments are stored in %rdi, %rsi, %rdx, %rcd, %r8, %r9
 - Return val is stored in %rax
 - Caller vs. callee save registers